

PORTADA

Análisis y Diseño de Sistemas II

Semana 14 — Patrones GoF de Comportamiento II: State, Template Method, Iterator

Universidad de Antioquia · Ingeniería de Sistemas



APERTURA

Ayer eligieron un algoritmo, reaccionaron a eventos y encapsularon una petición.

Hoy: comportamiento que cambia *solo*, pasos que se heredan, y recorridos que no exponen su estructura.

TRANSICIÓN

De tres patrones de comportamiento a los siguientes tres

Ayer, martes, vieron los tres primeros patrones de la familia de **comportamiento** de GoF: **Strategy** (encapsula un algoritmo intercambiable que el cliente elige), **Observer** (notifica automáticamente a los objetos dependientes cuando cambia el estado de un sujeto) y **Command** (encapsula una petición como un objeto, para poder parametrizarla, encolarla o deshacerla). No se repite esa explicación hoy — se asume vista.

Hoy se cierra la familia con tres patrones más, que resuelven problemas distintos: un objeto cuyo comportamiento cambia **según su propio estado interno** (State), un algoritmo con una estructura fija pero pasos que varían entre subclases (Template Method), y una forma de recorrer una colección sin exponer cómo está construida por dentro (Iterator).

CONCEPTO

Recordando: la familia de comportamiento

Las tres familias del catálogo GoF son: creacional (semana 12), estructural (semana 13) y **de comportamiento** (semanas 13-14). Los patrones de **comportamiento** se ocupan de cómo se distribuyen responsabilidades y se comunican los objetos entre sí — no de cómo se crean ni de cómo se ensamblan en estructuras más grandes, sino de **cómo colaboran en tiempo de ejecución**.

- El catálogo GoF original define **11 patrones de comportamiento** en total.
- Ayer se vieron 3: **Strategy, Observer, Command**.
- Hoy se ven 3 más: **State, Template Method, Iterator** — 6 de los 11, sin pretender agotar el catálogo completo.

TRANSICIÓN

Primer problema: un objeto que se comporta distinto según su propio estado

Piensen en un pedido de una tienda en línea: puede estar *pendiente*, *enviado* o *cancelado*. Las operaciones `enviar()` y `cancelar()` deben comportarse de forma completamente distinta según en cuál de esos estados se encuentre el pedido en ese momento — y esa diferencia no depende de una elección externa del cliente, sino de la **historia interna del propio objeto**.

CONTRA EJEMPLO

Un atributo de texto y condicionales repetidos por todo el código

```
class Pedido:
    def __init__(self):
        self.estado = "pendiente"

    def enviar(self):
        if self.estado == "pendiente":
            self.estado = "enviado"
        elif self.estado == "enviado":
            raise Exception("Ya fue enviado")
        elif self.estado == "cancelado":
            raise Exception("Pedido cancelado")

    def cancelar(self):
        if self.estado == "pendiente":
            self.estado = "cancelado"
        elif self.estado == "enviado":
            raise Exception("No se puede cancelar")
```

Cada método nuevo (`reactivar()` , `reembolsar()` ...) repetirá la misma cadena de `if / elif` sobre `self.estado` . Agregar un estado nuevo obliga a tocar **todos** los métodos existentes, uno por uno.

CONCEPTO

Definición: State

State permite que un objeto altere su comportamiento cuando cambia su **estado interno**, dando la impresión de que el objeto cambiara de clase. Cada estado posible se modela como una **clase separada** que implementa una interfaz común, y el objeto original **delega** el comportamiento dependiente del estado al objeto-estado que tiene en ese momento — en vez de usar un atributo de tipo texto/enum con condicionales repetidos.



CONCEPTO

State, en código

```
class EstadoPedido:                                # interfaz común
    def enviar(self, pedido): ...
    def cancelar(self, pedido): ...

class EstadoPedidoPendiente(EstadoPedido):
    def enviar(self, pedido):
        pedido.estado = EstadoPedidoEnviado()
    def cancelar(self, pedido):
        pedido.estado = EstadoPedidoCancelado()

class EstadoPedidoEnviado(EstadoPedido):
    def cancelar(self, pedido):
        raise Exception("Ya fue enviado")

class Pedido:
    def __init__(self):
        self.estado = EstadoPedidoPendiente()
    def enviar(self):    self.estado.enviar(self)
    def cancelar(self): self.estado.cancelar(self)
```

`Pedido` ya no pregunta "¿en qué estado estoy?" — simplemente delega en `self.estado` . Agregar un estado nuevo significa agregar una clase nueva, sin tocar las que ya existen.

CONCEPTO · RELACIÓN CON LO VISTO

State y Strategy: se parecen en la estructura, no en la intención

State y Strategy (visto ayer) comparten la misma forma en el diagrama de clases: un objeto delega en una interfaz común implementada por varias clases. La diferencia está en **quién decide el cambio** y **por qué**:

- **Strategy**: el cliente **elige** el algoritmo concreto (ej. qué método de ordenamiento usar) y ese algoritmo normalmente **no cambia por sí solo** mientras el objeto vive.
- **State**: las transiciones ocurren **desde dentro** del propio objeto-estado (`EstadoPedidoPendiente.enviar()` es quien decide pasar a `EstadoPedidoEnviado`) — el objeto se mueve solo de un estado a otro a medida que ocurren eventos de su ciclo de vida.

TRANSICIÓN

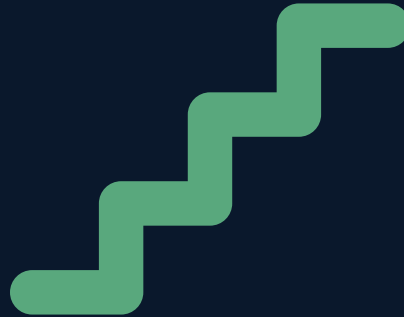
Segundo problema: un algoritmo con la misma forma, pero pasos distintos

State resuelve un objeto que cambia de comportamiento con el tiempo. El problema siguiente es distinto: varios algoritmos comparten exactamente la **misma secuencia de pasos** (leer datos, transformarlos, validarlos, exportarlos), pero uno o más de esos pasos concretos varían según el caso — y no se quiere duplicar la secuencia completa cada vez.

CONCEPTO

Definición: Template Method

Template Method define el **esqueleto de un algoritmo** en un método de una clase base —fijando el orden de los pasos—, delegando la implementación de algunos de esos pasos a las subclases mediante métodos que estas sobrescriben. La **estructura general** del algoritmo no cambia nunca; solo cambian los pasos concretos que cada subclase decide implementar distinto.



CONCEPTO

Template Method, en código

```
class ProcesadorDeReporte:          # clase base
    def procesar(self, datos):        # esqueleto fijo
        datos = self.leer(datos)
        datos = self.transformar(datos) # abstracto
        self.validar(datos)           # con default
        self.exportar(datos)          # abstracto

    def leer(self, datos):
        return datos

    def validar(self, datos):
        if not datos:
            raise ValueError("Reporte vacío")

    def transformar(self, datos): raise NotImplementedError
    def exportar(self, datos): raise NotImplementedError

class ReporteCSV(ProcesadorDeReporte):
    def transformar(self, datos):
        return [",".join(f) for f in datos]
    def exportar(self, datos):
        print("\n".join(datos))
```

CONCEPTO · RELACIÓN CON LO VISTO

Template Method vive de herencia y polimorfismo

En la **semana 7** vieron **herencia** (una subclase reutiliza y extiende lo definido en su clase base) y **polimorfismo** (el mismo mensaje, `transformar()` o `exportar()`, produce comportamientos distintos según la clase concreta que responde).

Template Method es exactamente ese mecanismo aplicado con disciplina: el método `procesar()` de la clase base **nunca se sobrescribe** —fija el orden—, y solo los pasos marcados como abstractos se dejan a criterio de cada subclase.

A diferencia de State, aquí no hay objetos delegados en tiempo de ejecución: la variación ocurre **por herencia**, en tiempo de diseño de la jerarquía de clases.

TENSIÓN CONCEPTUAL

El costo de fijar el algoritmo en una jerarquía de herencia

Template Method resuelve la duplicación de la secuencia de pasos, pero paga un precio: la variación queda **atada a la herencia**. Si en tiempo de ejecución se necesitara cambiar de "receta" dinámicamente —no solo elegirla al crear el objeto, sino intercambiarla sobre la marcha— la jerarquía de subclases se vuelve rígida para eso.

Ese es precisamente el escenario donde conviene **Strategy** (visto ayer) en lugar de Template Method: Strategy resuelve el mismo tipo de "paso variable" pero mediante **composición** (un objeto colaborador inyectado) en vez de herencia, permitiendo cambiar el comportamiento en tiempo de ejecución sin tocar la jerarquía de clases.

TRANSICIÓN

Tercer problema: recorrer una colección sin exponer cómo está construida

State y Template Method resuelven variación de comportamiento. El tercer problema de hoy es distinto: un cliente necesita recorrer, uno por uno, los elementos de una colección —que puede ser una lista, un árbol o cualquier otra estructura— **sin conocer los detalles** de cómo esa colección está construida por dentro.

CONCEPTO

Definición: Iterator

Iterator provee una forma de acceder secuencialmente a los elementos de una colección —lista, árbol u otra estructura— **sin exponer su representación interna**, a través de una interfaz común de recorrido: típicamente dos operaciones, `hayMas()` (¿queda algún elemento por visitar?) y `siguiente()` (entregar el próximo elemento y avanzar).



CONCEPTO

Iterator, en código

```
interface Iterador:
    def hayMas(self) -> bool: ...
    def siguiente(self): ...

class IteradorDeLista(Iterador):
    def __init__(self, elementos):
        self.elementos = elementos
        self.posicion = 0

    def hayMas(self):
        return self.posicion < len(self.elementos)

    def siguiente(self):
        elemento = self.elementos[self.posicion]
        self.posicion += 1
        return elemento

it = IteradorDeLista([10, 20, 30])
while it.hayMas():
    print(it.siguiente())
```

El cliente solo conoce `hayMas()` y `siguiente()` — no sabe (ni necesita saber) que por dentro hay un arreglo con un índice.

CONCEPTO

Por qué rara vez se implementa Iterator desde cero hoy

La mayoría de los lenguajes modernos ya integran este patrón **de forma nativa** en sus estructuras de datos: el `for / for-each` de Java, Python, C# o JavaScript sobre una lista, un diccionario o un árbol está usando, por dentro, exactamente esta idea —una interfaz común de recorrido que oculta la representación interna de la colección.

Por eso Iterator rara vez se escribe a mano en un proyecto actual: el lenguaje o su librería estándar ya lo resuelve. Aun así, entender el patrón importa: explica **por qué** esas estructuras nativas funcionan como funcionan, y sigue siendo relevante cuando se diseña una colección propia que debe poder recorrerse igual que las nativas.

INTERACCIÓN

¿State, Template Method o Iterator?

En el chat, para cada caso indiquen cuál de los tres patrones de hoy calza mejor:

1. Un reproductor multimedia cuyo botón "play/pausa" hace algo distinto según si el reproductor está *detenido*, *reproduciendo* o en *pausa*.
2. Un proceso de "generar informe" que siempre sigue los mismos cuatro pasos en el mismo orden, pero el paso de "dar formato a la salida" cambia según se exporte a PDF o a HTML.
3. Un menú de restaurante propio (no una lista nativa del lenguaje) sobre el que se necesita recorrer cada plato uno por uno, sin que el código cliente sepa si por dentro es un arreglo o una lista enlazada.

5 minutos · respondan en el chat

SÍNTESIS

Los 6 patrones de comportamiento vistos esta semana

Patrón	Idea central
Strategy	Encapsula un algoritmo intercambiable que el cliente elige.
Observer	Notifica automáticamente a los dependientes cuando cambia un sujeto.
Command	Encapsula una petición como objeto, para parametrizarla, encolarla o deshacerla.
State	Un objeto cambia de comportamiento según su propio estado interno.
Template Method	Esqueleto fijo de un algoritmo; las subclases completan pasos concretos.
Iterator	Recorrido secuencial de una colección sin exponer su estructura interna.

GoF define 11 patrones de comportamiento en total; esta semana se cubrieron 6, sin pretender agotar el catálogo completo.



CIERRE

El viernes: antipatronos y laboratorio de cierre de la Unidad 5

Con los seis patrones de comportamiento, más los patrones creacionales y estructurales de las semanas 12 y 13, se completa el recorrido por el catálogo GoF de esta unidad. El **viernes** se cierra la **Unidad 5 (Patrones de Diseño y Antipatronos)** con dos bloques: una revisión de **antipatronos de diseño** —las formas comunes en que estos mismos problemas se resuelven mal— y un **laboratorio de cierre de unidad** aplicando lo visto al proyecto de curso.

